

GK'SE — Great Knights' Search Engine

Documentação Técnica do Projeto

Projeto de Estruturas de Dados e Algoritmos — FGV

2020

Sumário

1	Visão Geral	3
1.1	Funcionalidades Principais	3
1.2	Escopo dos Dados	3
2	Arquitetura do Sistema	4
2.1	Visão Geral da Arquitetura	4
2.2	Estrutura de Diretórios	4
3	Estruturas de Dados	6
3.1	Árvore Trie	6
3.1.1	Estrutura do Nó (Node)	6
3.1.2	Classe Trie	6
3.2	Classe Pair	6
3.3	Classe str_dt	6
4	Algoritmos Principais	7
4.1	Conversão de Caracteres (convert)	7
4.2	Inserção na Trie	7
4.3	Busca (find)	7
4.4	Interseção Ordenada (inter2sorted)	7
4.5	Sugestão de Palavras (Levenshtein)	7
4.6	Serialização e Desserialização	8
4.6.1	Formato de Serialização	8
4.6.2	Desserialização (ShootingChars)	8
5	Pipeline de Pré-Processamento	9
5.1	Etapa 1: Extração de Palavras Únicas (get_words.py)	9
5.2	Etapa 2: Extração de Títulos (aMaker.py)	9
5.3	Etapa 3: Extração de Textos (aTaker.py)	9
5.4	Etapa 4: Serialização da Trie (trieSerial.cpp)	9
6	Interface de Linha de Comando	10
6.1	Fluxo de Uso	10
6.2	Exemplo de Uso	10

7	Interface Web (Servidor HTTP)	11
7.1	Tecnologias	11
7.2	Endpoints	11
7.3	Formato de Resposta	11
7.4	Compilação e Execução	11
8	Complexidade Computacional	12
8.1	Uso de Memória	12
9	Pré-Requisitos e Dependências	13
9.1	Para Executar o Motor de Busca (Terminal)	13
9.2	Para o Servidor Web	13
9.3	Para Reprocessar Dados	13
10	Instruções de Instalação e Execução	14
10.1	Modo Terminal (Recomendado)	14
10.2	Modo Web	14
11	Glossário	15
12	Créditos	15

1 Visão Geral

O **GK'SE (Great Knights' Search Engine)** é um motor de busca textual desenvolvido como projeto acadêmico para a disciplina de Estruturas de Dados e Algoritmos da FGV (Fundação Getulio Vargas).

O sistema indexa o corpus **WikiCorpus** (textos em inglês da Wikipedia) utilizando uma **árvore Trie** como estrutura de dados principal. Permite buscas por palavras únicas ou múltiplas palavras, retornando artigos relevantes ordenados por frequência de ocorrência. Quando uma palavra não é encontrada, o sistema sugere alternativas utilizando o algoritmo de **distância de Levenshtein**.

1.1 Funcionalidades Principais

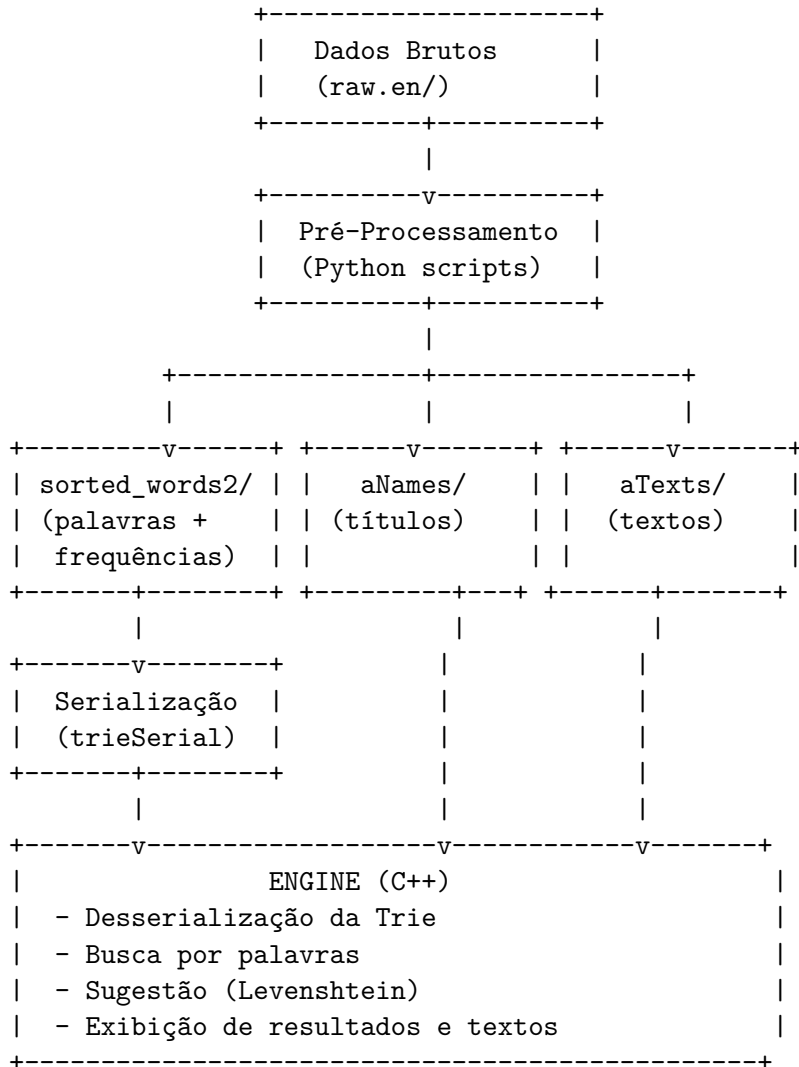
- **Busca por palavra única:** localiza todos os documentos que contêm a palavra pesquisada.
- **Busca por múltiplas palavras:** realiza a interseção dos resultados de cada palavra para encontrar documentos que contenham todas as palavras.
- **Ranqueamento por frequência:** os resultados são ordenados pela frequência com que a palavra aparece em cada documento.
- **Sugestão de palavras:** caso a palavra não seja encontrada, sugere palavras similares usando distância de Levenshtein.
- **Visualização de textos:** permite abrir e ler o conteúdo completo de um artigo diretamente no terminal.
- **Interface web** (opcional): servidor HTTP em C++ com página HTML para buscas via navegador.
- **Serialização/Desserialização:** a árvore Trie pode ser salva em arquivo e reconstruída sem reprocessar os dados brutos.

1.2 Escopo dos Dados

- **Corpus:** WikiCorpus (raw.en) — textos em inglês extraídos da Wikipedia.
- **Volume:** 164 arquivos de texto bruto, divididos em blocos de 10.000 documentos cada.
- **Total aproximado:** mais de 1.600.000 documentos indexados.

2 Arquitetura do Sistema

2.1 Visão Geral da Arquitetura



2.2 Estrutura de Diretórios

Diretório/Arquivo	Descrição
<code>engine.cpp</code>	Programa principal — interface de busca no terminal
<code>trie.cpp</code>	Implementação da árvore Trie, serialização/desserialização, e busca com Levenshtein
<code>trie_builder.cpp</code>	Construção da Trie a partir dos arquivos de palavras pré-processados
<code>Pair.cpp</code>	Classe auxiliar <code>Pair</code> (id, frequência) e classe <code>str_dt</code> para sugestões

Diretório/Arquivo	Descrição
<code>trieSerial</code>	Arquivo binário da árvore serializada (gerado/baixado)
<code>aNames/</code>	Títulos dos artigos organizados por faixa de índice
<code>aTexts/</code>	Textos completos dos artigos organizados por faixa de índice
<code>DataTreatment/</code>	Scripts Python para pré-processamento dos dados brutos
<code>Serialization/</code>	Scripts para serialização/desserialização da Trie
<code>Search_engine-servidor/</code>	Servidor HTTP com interface web

3 Estruturas de Dados

3.1 Árvore Trie

A estrutura de dados central é uma **Trie** (árvore de prefixos) com 36 filhos por nó, suportando:

- **10 dígitos:** 0-9 (índices 0-9)
- **26 letras:** a-z (índices 10-35)

3.1.1 Estrutura do Nó (Node)

```
struct Node {  
    Node *aChild[36];           // 36 ponteiros para filhos  
    vector<Pair> docs;           // lista de pares (id_documento, frequência)  
};
```

Cada nó terminal (fim de palavra) possui um vetor `docs` que armazena os pares (`id_documento`, `frequência`) de todos os documentos onde aquela palavra aparece.

3.1.2 Classe Trie

```
class Trie {  
    Node *aRoot;                // raiz da árvore  
public:  
    Node* insert(string aWord);   // insere uma palavra  
    vector<Pair> find(string aPhrase); // busca uma frase  
    vector<str_dt> suggest(string, int, int); // sugere palavras similares  
    void Serialize();             // salva em arquivo  
    void Deserialize();           // carrega de arquivo  
};
```

3.2 Classe Pair

Representa um par (`id_documento`, `frequência`):

```
class Pair {  
    int core[2];                // core[0] = id, core[1] = frequência  
};
```

O operador `<` é invertido (ordena por frequência decrescente) para que `std::sort` retorne os documentos mais relevantes primeiro.

3.3 Classe str_dt

Auxiliar para sugestões de palavras:

```
class str_dt {  
    string str;                 // palavra sugerida  
    int len;                    // quantidade de documentos  
};
```

4 Algoritmos Principais

4.1 Conversão de Caracteres (**convert**)

Converte uma string em vetor de índices (0-35):

- Dígitos '0'-'9' são mapeados para índices 0-9
- Letras minúsculas 'a'-'z' são mapeadas para índices 10-35
- Letras maiúsculas 'A'-'Z' são mapeadas para índices 10-35 (case-insensitive)
- Caracteres especiais e acentuados são ignorados

4.2 Inserção na Trie

A inserção percorre os índices convertidos da palavra, criando nós filhos quando necessário. Retorna um ponteiro ao nó terminal para que os pares (**id**, **frequência**) possam ser inseridos.

4.3 Busca (**find**)

Para uma frase com múltiplas palavras:

1. Divide a frase em palavras individuais.
2. Busca cada palavra na Trie, obtendo o vetor de pares (**id**, **freq**).
3. Realiza a **interseção** dos vetores usando a função **inter2sorted**, que opera em tempo linear aproveitando que os documentos estão ordenados por ID.

4.4 Interseção Ordenada (**inter2sorted**)

Realiza merge-intersect de dois vetores ordenados por ID de documento:

- Percorre ambos os vetores simultaneamente com dois ponteiros.
- Quando os IDs coincidem, o par é incluído no resultado.
- Complexidade: $O(n + m)$ onde n e m são os tamanhos dos vetores.

4.5 Sugestão de Palavras (**Levenshtein**)

Quando uma palavra não é encontrada, o sistema sugere alternativas usando **distância de edição de Levenshtein** integrada à travessia da Trie:

1. Inicializa a primeira linha da matrix de programação dinâmica.
2. Percorre recursivamente os filhos da raiz da Trie.
3. Em cada nó, calcula o custo de inserção, deleção e substituição.
4. Se a distância no último elemento da linha for menor ou igual ao custo máximo (**maxCost=2**), e o nó tiver documentos, a palavra é adicionada às sugestões.
5. A recursão é podada quando o mínimo da linha corrente excede **maxCost**.
6. Limita-se a no máximo 25.000 sugestões para evitar travamento.

Referências:

- Steve Hanov: *“Fast and Easy Levenshtein distance using a Trie”*
- Murilo Adriano: *“Fast and easy Levenshtein distance using a trie in C++”*

4.6 Serialização e Desserialização

4.6.1 Formato de Serialização

A árvore é serializada em uma string compacta com os seguintes símbolos:

Símbolo	Significado
+	Próximo filho existe — descer para ele
n	Próximo filho é nulo — pular
/	Fim dos dados do nó (sem documentos se imediatamente após filhos)
id,freq,	Par (id_documento, frequência) separado por vírgulas

4.6.2 Desserialização (ShootingChars)

Percorre a string serializada caractere a caractere, reconstruindo a árvore:

1. Para cada posição de filho (0-35), verifica se é **+** (criar nó) ou **n** (pular).
2. Ao encontrar **/**, retorna (nó sem documentos).
3. Caso contrário, lê os pares **id,freq** até encontrar **/**.

5 Pipeline de Pré-Processamento

5.1 Etapa 1: Extração de Palavras Únicas (`get_words.py`)

Entrada: Corpus bruto (`raw.en/`)

Processo:

1. Para cada arquivo do corpus, extrai todas as palavras (letras e números apenas).
2. Remove acentos usando normalização Unicode (NFKD).
3. Constrói um dicionário em memória onde:
 - Chave = palavra
 - Valor = dicionário {id_documento: frequência}
4. Salva em arquivos de texto na pasta `words/` (blocos de 50.000 palavras).
5. Ordena os IDs de documentos por frequência decrescente usando Merge Sort e salva em `sorted_words/`.

Saída: Pasta `sorted_words/` (ou `sorted_words2/`) com 64 arquivos. Formato:

```
$palavra  
id1 freq1 id2 freq2 ...
```

Requisito: ~16GB de RAM para processar o corpus completo.

5.2 Etapa 2: Extração de Títulos (`aMaker.py`)

Entrada: Corpus bruto (`raw.en/`)

Processo: Extrai os títulos e IDs (campo `dbindex`) das tags `<doc>` de cada artigo.

Saída: Pasta `aNames/` com 164 arquivos (um por bloco de 10.000 documentos).

5.3 Etapa 3: Extração de Textos (`aTaker.py`)

Entrada: Corpus bruto (`raw.en/`)

Processo: Extrai o conteúdo textual de cada artigo, marcando o início com `"Marked: low fidelity"` seguido do ID.

Saída: Pasta `aTexts/` com 164 arquivos.

5.4 Etapa 4: Serialização da Trie (`trieSerial.cpp`)

Entrada: Pasta `sorted_words/`

Processo:

1. Constrói a Trie inserindo todas as 3.200.000+ palavras.
2. Serializa a árvore em um único arquivo `trieSerial`.

Saída: Arquivo `trieSerial` (~centenas de MB).

Requisito: ~9GB de RAM.

6 Interface de Linha de Comando

6.1 Fluxo de Uso

1. **Inicialização:** O programa carrega a Trie desserializando o arquivo `trieSerial` (2-4 minutos).
2. **Busca:** O usuário digita uma consulta (uma ou mais palavras).
3. **Resultados:** Exibe até 20 resultados por vez, com título e opção de ver mais.
4. **Abertura de artigo:** O usuário pode digitar o número de um resultado para ler o texto completo.
5. **Sugestões:** Se a palavra não for encontrada:
 - Para buscas de palavra única, sugere palavras similares.
 - Para múltiplas palavras, informa que sugestões múltiplas ainda não são suportadas.
6. **Saída:** Digitar 0 encerra o programa.

6.2 Exemplo de Uso

```
-> Deserialization finished in 180.5 seconds!
-> Type your query: artificial intelligence

-> Finding: artificial intelligence...
-> 1523 results found in 0.002 seconds (2.1 ms).
-> Results:
  > [1] Artificial intelligence
  > [2] History of artificial intelligence
  > [3] Machine learning
  ...

-> Do you want to open any result [n or result number]? 1
(texto completo do artigo é exibido)

-> More results [y/n]? n

-> (To exit, enter the number 0)
```

7 Interface Web (Servidor HTTP)

7.1 Tecnologias

- **Backend:** Servidor HTTP em C++ usando a biblioteca **Simple-Web-Server** (header-only).
- **Dependência:** Boost C++ (filesystem, thread, system).
- **Frontend:** HTML + JavaScript simples com `fetch` API.

7.2 Endpoints

Rota	Método	Descrição
/index.html	GET	Página principal com formulário de busca
/query?text={texto}	GET	Realiza a busca e retorna JSON com resultados
/[arquivo]	GET	Serve arquivos estáticos da pasta <code>web/</code>

7.3 Formato de Resposta

```
{"res": "titulo1<BR>titulo2<BR>..."}
```

7.4 Compilação e Execução

```
cd Search_engine-servidor/  
g++ servidor.cpp -I /usr/include \  
    -lboost_filesystem -lboost_thread \  
    -lpthread -lboost_system -o servidor  
./servidor  
# Acessar: http://localhost:8080/index.html
```

8 Complexidade Computacional

Operação	Complexidade
Inserção de palavra na Trie	$O(k)$ onde k = comprimento da palavra
Busca de palavra única	$O(k)$
Interseção de resultados	$O(n + m)$ onde n, m = tamanhos dos vetores
Busca com múltiplas palavras	$O(k_1 + k_2 + \dots + n + m + \dots)$
Sugestão (Levenshtein)	$O(N \cdot k \cdot w)$ no pior caso (N=nós, k=tam. palavra, w=largura)
Serialização	$O(N)$ onde N = total de nós
Desserialização	$O(S)$ onde S = tamanho do arquivo serializado

8.1 Uso de Memória

- **Nó da Trie:** 36 ponteiros (288 bytes em 64-bit) + vetor de Pairs.
- **Total estimado em execução:** 4-9 GB de RAM dependendo do corpus carregado.
- **Arquivo trieSerial:** centenas de MB.

9 Pré-Requisitos e Dependências

9.1 Para Executar o Motor de Busca (Terminal)

Requisito	Versão Mínima
g++ (GCC)	7+ (suporte a C++11)
RAM	4 GB mínimo
Dados pré-processados	<code>trieSerial</code> , <code>aNames/</code> , <code>aTexts/</code>

9.2 Para o Servidor Web

Requisito	Versão Mínima
Boost C++	1.65+
Pacotes Boost necessários	<code>filesystem</code> , <code>thread</code> , <code>system</code>

9.3 Para Reprocessar Dados

Requisito	Versão
Python	3.7+
RAM	16 GB recomendado
Corpus WikiCorpus	<code>raw.en</code> (~3 GB compactado)

10 Instruções de Instalação e Execução

10.1 Modo Terminal (Recomendado)

```
# 1. Clonar ou baixar o repositório
git clone <url-do-repositorio>
cd Project_GKSearchEngine

# 2. Baixar dados pré-processados do Google Drive:
# https://drive.google.com/file/d/1C1REE2hm8JPc5KaK1YxJmO_M29VrZB8-
# Extrair na raiz do projeto (trieSerial, aNames/, aTexts/)

# 3. Compilar
g++ engine.cpp -o engine

# 4. Executar (aguardar 2-4 min para desserialização)
./engine
```

10.2 Modo Web

```
# 1. Instalar Boost
sudo apt install libboost-all-dev    # Ubuntu/Debian

# 2. Compilar servidor
cd Search_engine-servidor/
g++ servidor.cpp -I /usr/include \
    -lboost_filesystem -lboost_thread \
    -lpthread -lboost_system -o servidor

# 3. Executar
./servidor

# 4. Abrir no navegador: http://localhost:8080/index.html
```

11 Glossário

Termo	Definição
Trie	Árvore de prefixos; cada nó representa um caractere e caminhos da raiz às folhas formam palavras
Serialização	Processo de converter a estrutura da Trie em uma representação textual para armazenamento em disco
Desserialização	Processo inverso — reconstruir a Trie a partir do arquivo serializado
Levenshtein	Métrica de distância entre duas strings, definida pelo número mínimo de edições (inserção, deleção, substituição)
WikiCorpus	Corpus linguístico baseado na Wikipedia, disponibilizado pela UPC Barcelona
dbindex	Identificador único de cada documento/artigo no corpus
Pair	Classe que armazena o par (id_documento, frequência)
inter2sorted	Função de interseção de dois vetores ordenados

12 Créditos

- **Serialização/Desserialização:** Jorge Costa
- **Biblioteca Simple-Web-Server:** Ole Christian Eidheim
- **Corpus WikiCorpus:** NLP Group, UPC Barcelona
- **Referências para Levenshtein na Trie:** Steve Hanov, Murilo Adriano